Mapa del código TSSA

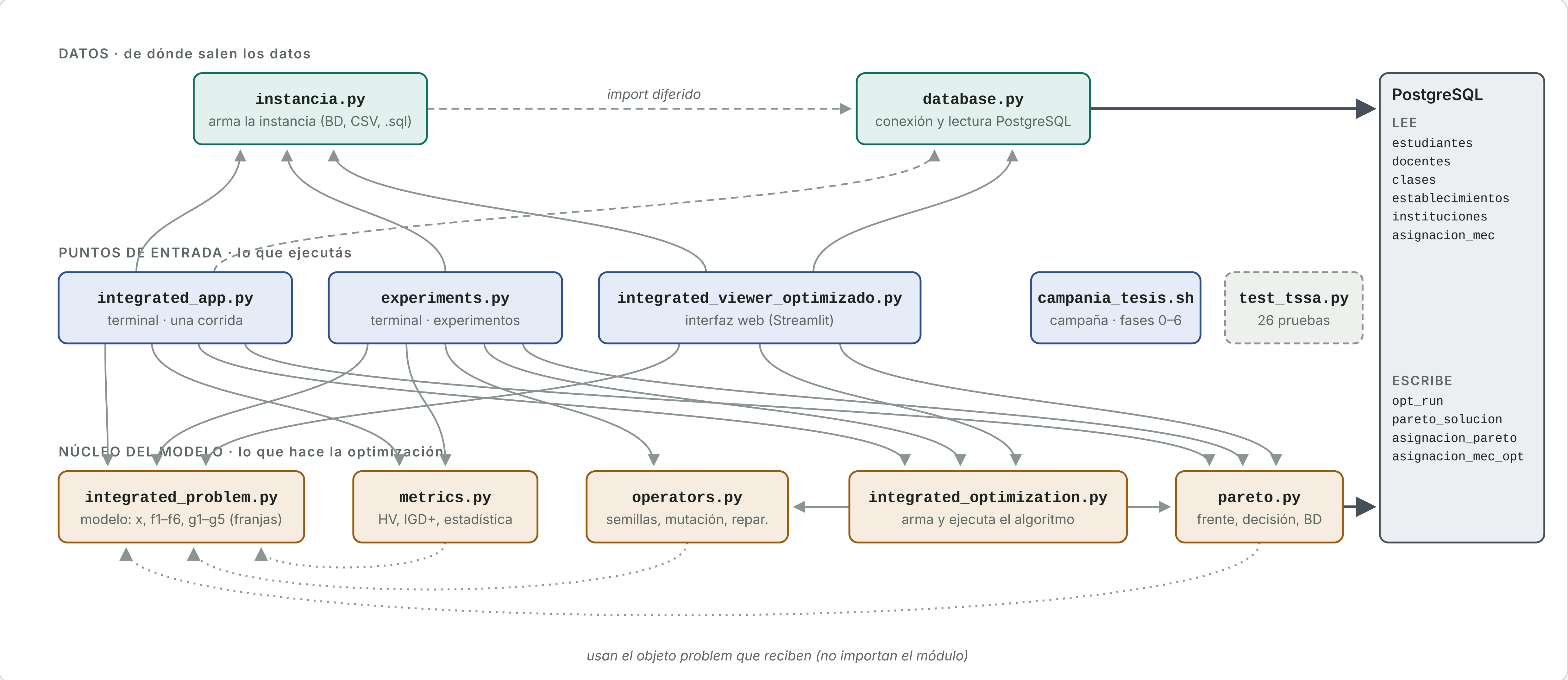
Cómo se conectan los 11 archivos Python del proyecto *Asignación Simultánea de Estudiantes y Docentes* y el script de la campaña experimental (versión del 2 de noviembre de 2026). Hay tres capas: lo que ejecutás, el núcleo del modelo y los datos. Sólo dos archivos tocan PostgreSQL.

Quién importa a quién

Cada flecha va del archivo que importa hacia el importado (`A → B`: A hace `from B import …`). Tocá o pasá el mouse sobre un archivo para ver sólo sus conexiones y su ficha abajo.

☐ punto de entrada ☐ núcleo del modelo ☐ datos — importa ---- import diferido (sólo si hace falta) ······ usa el objeto `problem` ---- la campaña lo ejecuta

— al seleccionar: lo que importa — quién lo importa


DATOS · de dónde salen los datos
instancia.py
arma la instancia (BD, CSV, .sql)
import diferido
database.py
conexión y lectura PostgreSQL
PostgreSQL
LEE
estudiantes
docentes
clases
establecimientos
instituciones
asignacion_mec
ESCRIBE
opt_run
pareto_solucion
asignacion_pareto
asignacion_mec_opt
PUNTOS DE ENTRADA · lo que ejecutás
integrated_app.py
terminal · una corrida
experiments.py
terminal · experimentos
integrated_viewer_optimizado.py
interfaz web (Streamlit)
campania_tesis.sh
campaña · fases 0–6
test_tssa.py
26 pruebas
NÚCLEO DEL MODELO · lo que hace la optimización
integrated_problem.py
modelo: x, f1–f6, g1–g5 (franjas)
metrics.py
HV, IGD+, estadística
operators.py
semillas, mutación, repar.
integrated_optimization.py
arma y ejecuta el algoritmo
pareto.py
frente, decisión, BD
usan el objeto problem que reciben (no importan el módulo)

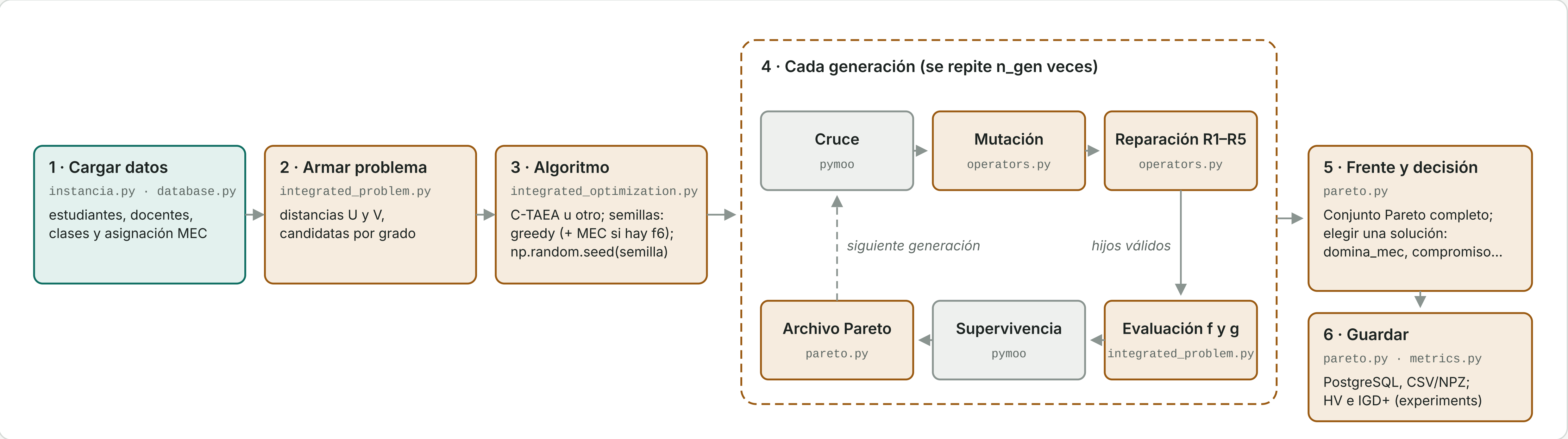
Los tres programas Python importan el núcleo (abajo) y la capa de datos (arriba); `campania_tesis.sh` no importa nada: ejecuta las pruebas, `experiments.py` e `integrated_app.py` por fases (sus flechas y las de las pruebas aparecen al seleccionarlos). `integrated_problem.py` no depende de ningún otro archivo del proyecto: los demás reciben el objeto `problem` y usan sus matrices. PostgreSQL se lee por `database.py` y se escribe por `pareto.py`.

Elegí un archivo

Tocá cualquier caja del mapa (o usá Tab y Enter) para ver qué hace, qué importa y quién lo importa. Empezá por `integrated_optimization.py`, el que conecta todo.

Qué pasa durante una corrida

El mismo código visto en el orden en que se ejecuta, por ejemplo con `python integrated_app.py`. El paso 4 es el ciclo que pymoo repite en cada generación; ahí se reparte casi todo el tiempo de cómputo.


1 · Cargar datos
instancia.py · database.py
estudiantes, docentes, clases y asignación MEC
2 · Armar problema
integrated_problem.py
distancias U y V, candidatas por grado
3 · Algoritmo
integrated_optimization.py
C-TAEA u otro; semillas: greedy (+ MEC si hay f6); np.random.seed(semilla)
4 · Cada generación (se repite n_gen veces)
Cruce
pymoo
Mutación
operators.py
Reparación R1–R5
operators.py
siguiente generación
hijos válidos
Archivo Pareto
pareto.py
Supervivencia
pymoo
Evaluación f y g
integrated_problem.py
5 · Frente y decisión
pareto.py
Conjunto Pareto completo; elegir una solución: domina_mec, compromiso...
6 · Guardar
pareto.py · metrics.py
PostgreSQL, CSV/NPZ; HV e IGD+ (experiments)

Pasos 1 a 3: se preparan una sola vez; la semilla fija también `np.random`, que usan los operadores propios, así que cada corrida es reproducible. Paso 4: en cada generación los hijos se cruzan, mutan, se reparan para cumplir R1–R5, se evalúan y sobreviven los mejores; las soluciones factibles se acumulan en el archivo Pareto. Pasos 5 y 6: al terminar se arma el Conjunto Pareto, se elige una solución y se guarda. Con `domina_mec` se elige la de compromiso entre las que mejoran a la asignación del MEC en todos los objetivos.

Referencia rápida

ARCHIVO	QUÉ HACE	IMPORTA DEL PROYECTO	LO IMPORTAN
`campania_tesis.sh`	Script bash que orquesta la campaña experimental completa y reanudable: 0 pruebas, 1 conexión y snapshot de la BD, 2 piloto de convergencia, 3 experimento principal (con el diseño 2×2), 4 estabilidad (complementaria), 5 ablaciones y 6 recomendación con C-TAEA en la BD. No importa módulos: ejecuta los programas Python.	—	—

`integrated_app.py`	Ejecuta UNA optimización desde la terminal: carga los datos, arma el problema, corre el algoritmo, muestra el Conjunto Pareto y lo compara con la asignación actual del MEC. Guarda en la BD o exporta a CSV/NPZ. Es la fase 6 de la campaña (C-TAEA con domina_mec).	`instancia.py`
		`database.py`
		`integrated_optimization.py`
		`integrated_problem.py`
		`metrics.py`
		`pareto.py`
			—

`experiments.py`	Protocolo experimental de la tesis: 30 corridas por algoritmo con semillas reproducibles, variantes algoritmo@k (por ejemplo ctaea@3), greedy como línea de base (excluido si es infactible), semillas de estabilidad cuando f6 es objetivo, indicadores (HV, IGD+, GD+), estadística y figuras.	`instancia.py`
		`integrated_optimization.py`
		`integrated_problem.py`
		`metrics.py`
		`operators.py`
		`pareto.py`
			`tests/test_tssa.py`

`integrated_viewer_optimizado.py`	Interfaz web con tres pestañas: asignación actual (objetivos del MEC), optimización (Conjunto Pareto, elección del método de decisión, incluido domina_mec, y guardado) y comparación antes/después con los mismos objetivos que integrated_app.py, más mapas y exportación a Excel.	`database.py`
		`instancia.py`
		`integrated_problem.py`
		`integrated_optimization.py`
		`pareto.py`
			—

`integrated_optimization.py`	Arma el algoritmo evolutivo (NSGA-II, NSGA-III, U-NSGA-III, C-TAEA, RVEA, SPEA2) con los operadores del TSSA, fija la semilla de la corrida (pymoo y np.random), inyecta las semillas de estabilidad si f6 es objetivo, lo ejecuta con pymoo, reúne el Conjunto Pareto y abre la conexión para guardarlo (DatabaseManager).	`operators.py`
		`pareto.py`
			`integrated_app.py`
			`experiments.py`
			`integrated_viewer_optimizado.py`
			`tests/test_tssa.py`

`integrated_problem.py`	Define el problema: el vector x = [XA | XD], los objetivos f1–f6 y las restricciones g1–g5. R4 (g4) se evalúa sobre franjas horarias: una clase Mañana-Tarde ocupa el día completo del docente. Precalcula las distancias U y V y las clases candidatas de cada alumno. No importa ningún otro archivo del proyecto.	—	`integrated_app.py`
			`experiments.py`
			`integrated_viewer_optimizado.py`
			`tests/test_tssa.py`

`operators.py`	Operadores propios del problema: población inicial por grado, mutación categórica, reparación R1–R5 (con franjas horarias), la heurística greedy y las semillas de estabilidad (MEC y mezclas MEC/greedy) para f6. Recibe el objeto problem y usa sus matrices y candidatas.	—	`experiments.py`
			`tests/test_tssa.py`
			`integrated_optimization.py`

`pareto.py`	Todo lo que pasa con el Conjunto Pareto: archivo externo por generación (tope de 2.000 soluciones), extracción del frente, decisión a posteriori (compromiso, rodilla, pseudo-pesos, domina_mec…), traducción a filas de asignación y guardado en CSV/NPZ y PostgreSQL (COPY).	—	`integrated_app.py`
			`experiments.py`
			`integrated_viewer_optimizado.py`
			`tests/test_tssa.py`
			`integrated_optimization.py`

`metrics.py`	Mide la calidad de los frentes (HV, IGD+, GD+, spacing), compara contra el MEC y el greedy (dominancia, % de mejora) y hace la estadística (Kruskal–Wallis, Mann–Whitney con Holm, A12).	—	`integrated_app.py`
			`experiments.py`
			`tests/test_tssa.py`

`instancia.py`	Arma la instancia (estudiantes, docentes, clases y asignación MEC) desde la BD, un snapshot CSV (datos_asu/), el script asignacion_mec.sql o una instancia sintética. Convierte la asignación del MEC en un vector x, asignando cada sección a una clase según su orden dentro de (establecimiento, grado, turno).	`database.py`	`integrated_app.py`
			`experiments.py`
			`integrated_viewer_optimizado.py`
			`tests/test_tssa.py`

`database.py`	Única configuración de conexión (DB_CONFIG, desde .env) y el engine de SQLAlchemy. Lee las tablas base de PostgreSQL. La campaña usa test_conexion() en la fase 1.	—	`integrated_app.py`
			`integrated_viewer_optimizado.py`
			`instancia.py`

`tests/test_tssa.py`	26 pruebas: ejemplo didáctico del preprint, regresión contra la v1.3 (tests/legacy), reparación, los 6 algoritmos, archivo Pareto, decisión, f4/f6, franjas Mañana-Tarde, mapeo de secciones, greedy infactible, reproducibilidad por semilla, semillas de estabilidad y domina_mec. Sus flechas se muestran sólo al seleccionarlo.	`instancia.py`
		`integrated_optimization.py`
		`integrated_problem.py`
		`operators.py`
		`metrics.py`
		`pareto.py`
		`experiments.py`
			—

`tests/legacy/original_integrated_problem.py` es la copia congelada de la v1.3 que sólo usa la prueba de regresión. `campania_tesis.sh` (bash) orquesta la campaña: 0 pruebas, 1 datos, 2 piloto, 3 experimento principal, 4 estabilidad, 5 ablaciones, 6 recomendación en la BD. Las dependencias externas principales son pymoo (algoritmos), pandas/numpy, SQLAlchemy y psycopg2 (base de datos) y Streamlit/Folium (viewer).